COMPUTER ENGINEERING

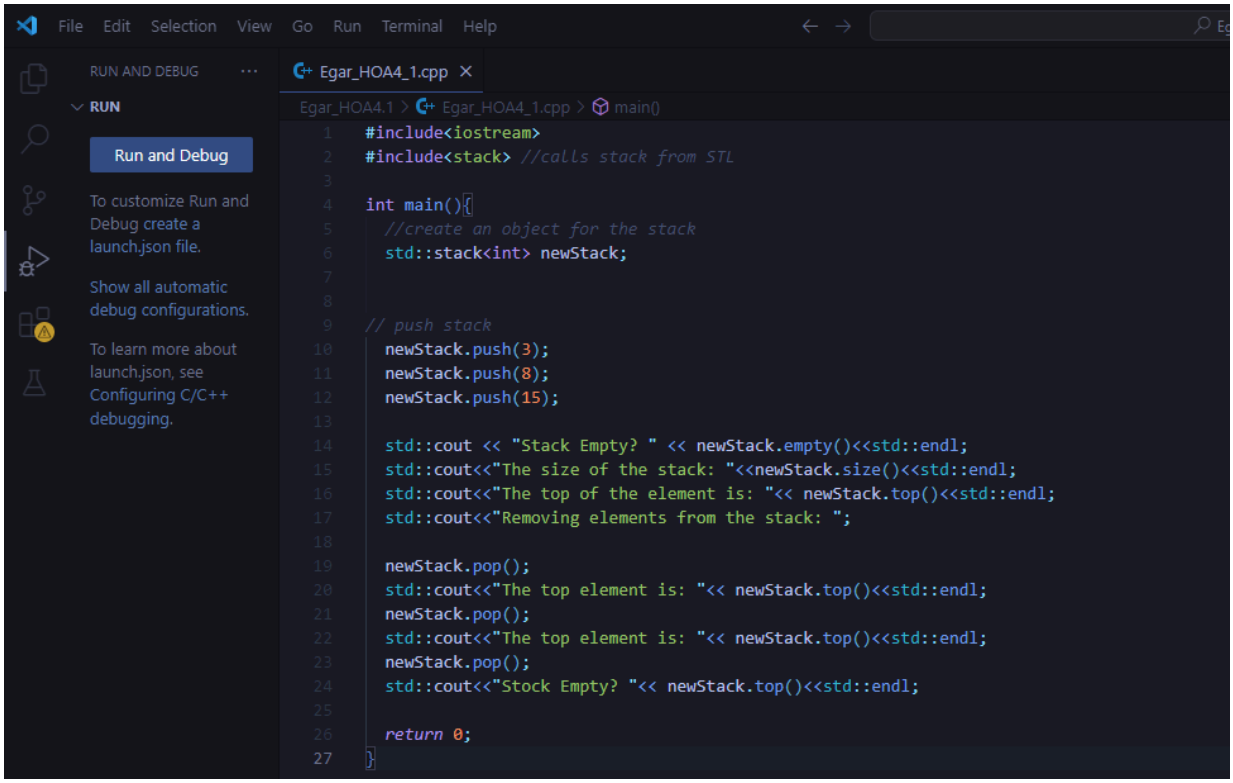
QUEZON CITY

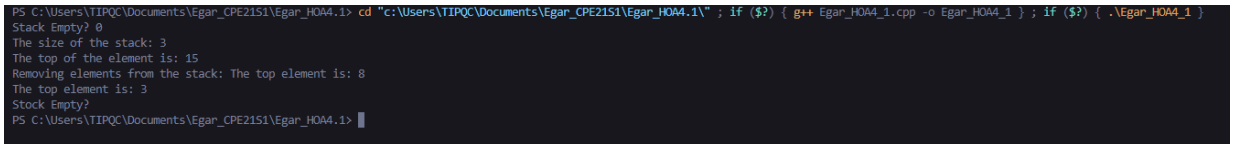
COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	07/28/2026	**DATE SUBMITTED:**	07/28/2026
NAME:	Jareld Dave R.Egar	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 4.1 Seatwork4.1 Array-Based vs STL stack

1. PROCEDURE OUTPUT

ILO A





ILO B part 1
Code

```cpp
#include <iostream>


//static array, what do we need?
const size_t maxCap = 100;
int stack[maxCap];
int top = -1, i, newData;
//always -1 is the top of the stack if it is empty


//functions
void push();
void pop();
void Top();
void display(); // addedfunction
bool isEmpty();
bool isFull();


int main() {
   int choice;
   std::cout << "Enter the number of max elements for the new stack: ";
   std::cin >> i;
   push();


   while(true) {
       std::cout << "stack operations" << std::endl;
       std::cout << "1. PUSH 2. POP 3. TOP 4. isEmpty 5. Display 6. Stop" << std::endl;
       std::cin >> choice;
       switch(choice) {
           case 1: push();
           break;
           case 2: pop();
           break;
           case 3: Top();
           break;
           case 4: std::cout << isEmpty() << std::endl;
           break;
           case 5: display(); // added function
           break;
            if(isEmpty()) std::cout << "Stack is empty" << std::endl;
            else std::cout << "Stack is not empty" << std::endl;
           break;
           case 6:
           return 0;

       }
```

```
    return 0;
}


bool isEmpty() {
    //create a condition to check whether the top element is equal to -1
    if (top == -1) return true;
    return false;
}


bool isFull() {
    //check if we reached the maximum capacity of the array
    if (top >= i-1) return true;
    return false;
}


void push() {
    //check if full -> if yes return an error
    if (isFull()) {
        std::cout << "stack overflow" << std::endl;
        return;
    }


    std::cout << "New Value to add into the stack" << std::endl;
    std::cin >> newData;


    //increment the top(indext by 1) then assign the new data;
    stack[++top] = newData;
}


void Top() {
    //check if the stack is empty
    if (isEmpty()) {
        std::cout << "stack underflow" << std::endl;
    }


    //
    std::cout << "The element on the top of the stack is " << stack[top] << std::endl;
}
```

```
void pop() {
    //check if the stack is empty
    if (isEmpty()) {
        std::cout << "stack underflow" << std::endl;
    }


    //display the top value
    std::cout << "popping" << stack[top] << std::endl;
    top--;
}

//added functions
void display(){

    std::cout << "\nStack"<<std::endl;
    for(int j = top; j != -1; j--) {
       std::cout << stack[j] << "\n";

       }
       std::cout<< std::endl;
    }
```

Output

```
Enter the number of max elements for the new stack: 2
New Value to add into the stack
1
stack operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. Display 6. Stop
1
New Value to add into the stack
2
stack operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. Display 6. Stop
5

Stack
2
1

stack operations
1. PUSH 2. POP 3. TOP 4. isEmpty 5. Display 6. Stop
6
PS C:\Users\TIPQC\Documents\Egar_CPE21S1\Egar_HOA4.1\Egar_HOA4.1ILOA>
```

DISCUSSION and CODE ANALYSIS: \

This activity helped me to understand how stack works by the help of arrays. In order for a stack to work you have to declare push(add value on top of the stack), pop (removing a stack) , and top (checking the value of the peek stack). Because of this lesson I learned that an array is used best to determine if the stack is full or not because an array is static or has a fixed value, unlike Linked List which is a dynamic type that also uses memory.

ILO B part 2

Main.cpp

main.cpp | Egar_ILOC_supple.cpp

Egar_HOA4.1 > ILOB2 > main.cpp > ...

```cpp
#include<iostream>
#include "stacklist.h"

int main(){

  std:: cout<< " --- PUSHING 'c' 'p' 'e' ONTO THE STACK --- \n";

  //push
  push('c');
  push('p');
  push('e');

  //display the current top
  Top<char>(); // Top should be e

  std::cout<<"\n----- Display all the elements ----- \n";

  displayAll<char>();

  std::cout<<"\n----- Popping all the elements ----- \n";

  while (!isEmpty<char>()){
    std::cout<< " Popped: " << pop<char>() << std::endl;

  }

  std::cout<<"\n ----- Testing the Underflow -----\n";
  pop<char>();

  return 0;
}
```

Stacklist.h

stacklist.h | ILOC.h

Egar_HOA4.1 > ILOB2 > stacklist.h > displayAll<T>()

```
#ifndef STACK_LIST_H
#define STACK_LIST_H

// class node that contains data and the next
template < typename T>
class Node{
  public:
  T data; //value that we need to store
  Node* next; // pointer to the next node

};

//global pointer
template <typename T>
  Node<T> *head = nullptr;

  //operations for creating the stack

  // push
  template <typename T>
  void push(T newData){
      //create a new node and assign the newData
      Node<T>* newNode = new Node<T>;
      newNode->data = newData;

      // new node points to current top, then becomes the new top

      newNode -> next = head<T>;
      head<T> = newNode;

  }

```

```cpp
// pop
template <typename T>
T pop(){
    // checks if the stack is empty
    if (head<T> == nullptr){
      std::cout << "Stack overflow. "<< std::endl;

    }

  // save the current top node
    Node<T>* temp = head<T>;
  // store data
    T tempVAL = temp->data;
  // move the head to the mext node
    head<T> = head<T> -> next;
  // free the removed node
    delete temp;
  // return the removed value
    return tempVAL;
}

```

```cpp
    // top
template<typename T>
void Top(){
    // error catching
    if(head<T> == nullptr){
        std::cout<<"Stack Underflow "<< std::endl;
        return;
    }
    //display
    std::cout<<"Top of stack: "<< head<T>->data <<std:: endl;
}

    // isEmpty
template<typename T>
bool isEmpty(){
    return head <T> == nullptr;


}

// display all
template <typename T>
void displayAll() {
    if(isEmpty<T>()) {
        std::cout << "stack is empty \n";
        return;
    }

    std::cout << " Stack contents " ;
    Node<T>* temp = head<T>;
    while (temp != nullptr){
        std::cout << "\n | "<< temp->data <<" | ";
        temp = temp->next;
    }
    std::cout << std::endl;
}

#endif
```

```
 --- PUSHING 'c' 'p' 'e' ONTO THE STACK ---
Top of stack: e

----- Display all the elements -----
 Stack contents
 | e |
 | p |
 | c |

----- Popping all the elements -----
 Popped: e
 Popped: p
 Popped: c

 ----- Testing the Underflow -----
Stack overflow.
```

DISCUSSION and CODE ANALYSIS:

This activity helped me understand the difference between implementing stacks using arrays and linked lists. Unlike array-based stacks, linked lists do not require an isFull() function because they can grow dynamically as long as memory is available. In my code, the head pointer always represents the top of the stack, making push() and pop() operations simple pointer updates. I also learned that checking if the stack is empty is important before removing or accessing elements to avoid errors. Overall, this activity improved my understanding of dynamic memory allocation, pointers, and the LIFO behavior of stacks implemented with linked lists.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

LO C: Solve problems using an implementation of stack:
The following problem definition and algorithm is provided for checking the balancing of symbols. Create an implementation using stacks. Your output must include the following:
a. Stack using Arrays
b. Stack using Linked Lists
c. (Optional) Stack using C++ STL
Problem Definition:
Stacks can be used to check whether the given expression has balanced symbols. This algorithm is very useful in compilers. Each time the parser reads one character at a time. If the character is an opening delimiter such as (, {, or [- then it is written to the stack. When a closing delimiter is encountered like), }, or]-the stack is popped. The opening and closing delimiters are then compared. If they match, the parsing of the string continues. If they do not match, the parser indicates that there is an error on the line.
Steps:
1. Create a Stack.
2. While(end of input is not reached) {
a) If the character read is not a symbol to be balanced, ignore it.
b) If the character is an opening symbol, push it onto the stack.
c) If it is a closing symbol:

i) Report an error if the stack is empty.
ii) Otherwise, pop the stack.
d) If the symbol popped is not the corresponding opening symbol, report an error.
3. At the end of input, if the stack is not empty: report an error.

Main.cpp

Egar_ILOC.cpp

Egar_Hoa4.1ILOC > Egar_ILOC.cpp > main()

```cpp
#include <iostream>
#include "ILOC.h"

int main() {

    std::string expression;
    char choice;

    do {
        std::cout << "Enter expression: ";
        std::getline(std::cin, expression);

        if (isBalanced(expression))
            std::cout << "\nExpression is Balanced.\n";
        else
            std::cout << "\nExpression is NOT Balanced.\n";

        std::cout << "\nDo you want to check another expression? (Y/N): ";
        std::cin >> choice;
        std::cin.ignore();

    } while (choice == 'Y' || choice == 'y');

    std::cout << "\nProgram Ended.\n";

    return 0;
}
```

Header File

Egar_Hoa4.1ILOC > C ILOC.h > push<T>(T)

```cpp
#ifndef ILOC_H
#define ILOC_H

#include <iostream>
#include <string>

template <typename T>
struct Node {
    T data;
    Node<T>* next;
};


template <typename T>
Node<T>* topNode = nullptr;

// Push
template <typename T>
void push(T value) {
    Node<T>* newNode = new Node<T>;
    newNode->data = value;
    newNode->next = topNode<T>;
    topNode<T> = newNode;

    std::cout << "Push: " << value << std::endl;
}

// Pop
template <typename T>
T pop() {
    if (topNode<T> == nullptr) {
        std::cout << "Stack Underflow!\n";
        return T();
    }
```

```
    Node<T>* temp = topNode<T>;
    T value = temp->data;
    topNode<T> = temp->next;
    delete temp;

    std::cout << "Pop: " << value << std::endl;

    return value;
}

// isEmpty
template <typename T>
bool isEmpty() {
    return topNode<T> == nullptr;
}

// clear the stack
template <typename T>
void clearStack() {
    while (!isEmpty<T>()) {
        pop<T>();
    }
}
```

```cpp
// BALANCED SYMBOLS FUNCTION
bool isBalanced(const std::string& expression) {

    clearStack<char>(); // reset stack

    for (int i = 0; i < expression.length(); i++) {

        char ch = expression[i];

        // Opening symbols
        if (ch == '(' || ch == '[' || ch == '{') {
            push<char>(ch);
        }

        // Closing symbols
        else if (ch == ')' || ch == ']' || ch == '}') {

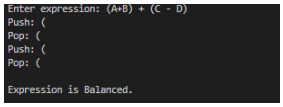
            if (isEmpty<char>()) {
                return false;
            }

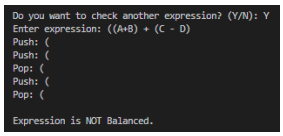
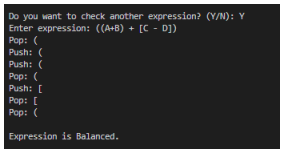
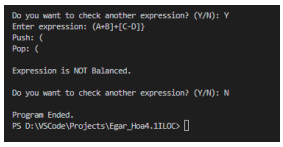
            char open = pop<char>();

            if ((ch == ')' && open != '(') ||
                (ch == ']' && open != '[') ||
                (ch == '}' && open != '{')) {
                return false;
            }
        }
    }

    return isEmpty<char>();
}

#endif
```

Expression	Valid (Y/N)	Output(Console Screenshot)	Analysis
(A+B) + (C - D)	Y		This is a valid expression because they have a closing and opening symbol

((A+B) + (C - D)	N		This is not valid because there is a missing ")" closing parenthesis
((A+B) + [C - D])	Y		This is valid because the parenthesis and brackets both have a opening and closing
((A+B]+[C-D]}	N		This is not valid because there is a missing closing parenthesis

DISCUSSION and CODE ANALYSIS:

This activity helped me understand the difference between implementing stacks using arrays and linked lists. Unlike array-based stacks, linked lists do not require an isFull() function because they can grow dynamically as long as memory is available. In my code, the head pointer always represents the top of the stack, making push() and pop() operations simple pointer updates. I also learned that checking if the stack is empty is important before removing or accessing elements to avoid errors.This activity helped me understand how stacks can be used to check whether an expression has balanced symbols using a linked list implementation. In the code, the stack is built dynamically using nodes, where each new symbol is added using the push() function and removed using the pop() function, following the Last In First Out principle. The program reads each character of the expression and pushes opening symbols like (, [, and {, while closing symbols trigger a pop operation and comparison to ensure correct matching. Based on the program results, expressions such as (A+B) + (C-D) and ((A+B) + [C-D]) are balanced because all symbols are properly matched and the stack is empty at the end. However, expressions like ((A+B) + (C-D) and (A+B)[C-D]} are not balanced because they either have missing closing symbols or mismatched pairs, which leaves elements in the stack or causes incorrect comparisons.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Fixing the grammar**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

COMPUTER ENGINEERING

QUEZON CITY

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.